



Windows Access Tokens

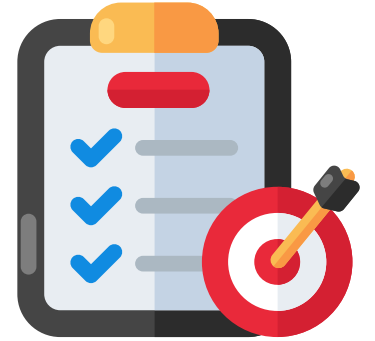
LuxCamp2025

Emanuel Duss <emanuel.duss@compass-security.com>

August 8, 2025

Welcome

- Agenda
 - Authentication on Windows
 - Logon Sessions
 - Logon Types
 - Cached Credentials
 - Access Tokens
 - User Impersonation (legit)
 - User Impersonation for Attackers
- Why
 - Pentest of Windows systems / Active Directory infrastructures
 - Better understanding of Windows authentication & access tokens
 - Can be used to take over user accounts (user impersonation)



Introduction

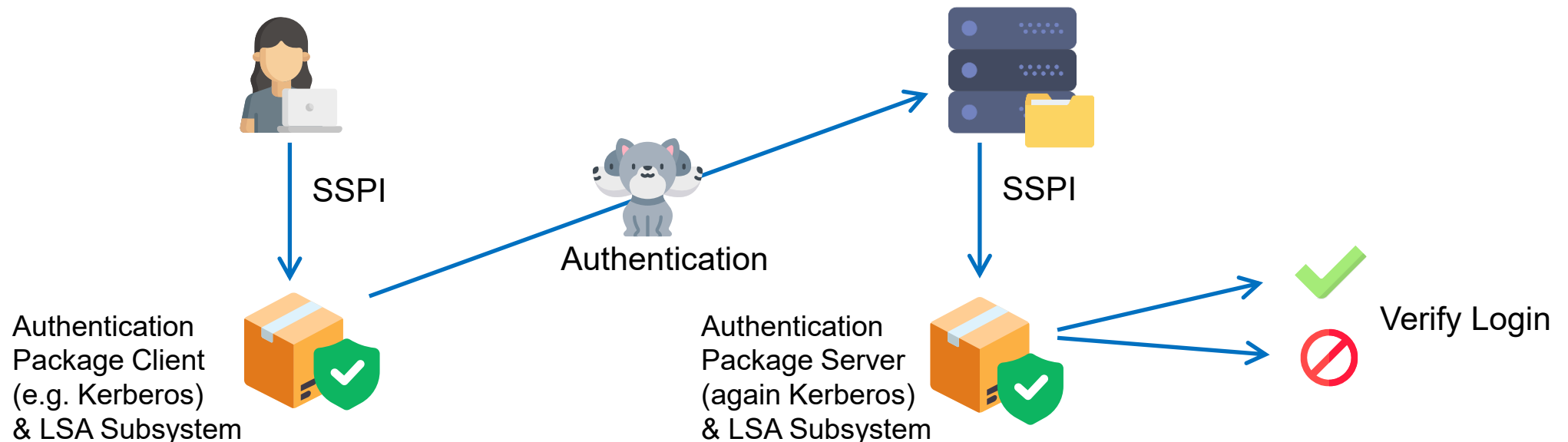
Windows Authentication

- Different components and protocols are involved during authentication
- Authentication Packages (APs) / Security Support Providers (SSPs)
 - Handle authentication & analyze login data
 - DLLs loaded by Local Security Authority (LSA) Subsystem
 - LSA Subsystem Service (LSASS, lsass.exe)
- An application can use an AP client to authenticate
- AP client sends credentials via SSP interface to AP server
- AP server authenticates client with credentials
 - Local Accounts authenticate via SAM
 - Domain Accounts authenticate via Active Directory



Windows Authentication

- Different APs for different protocols / methods
 - CredSSP (transmits username/password to target server via TLS)
 - NTLM
 - Kerberos
 - Negotiate (selects between NTLM and Kerberos)
 - Digest SSP / wDigest (challenge/response via HTTP/SASL, will be deprecated in the future)
 - MSV1_0 (local logins)

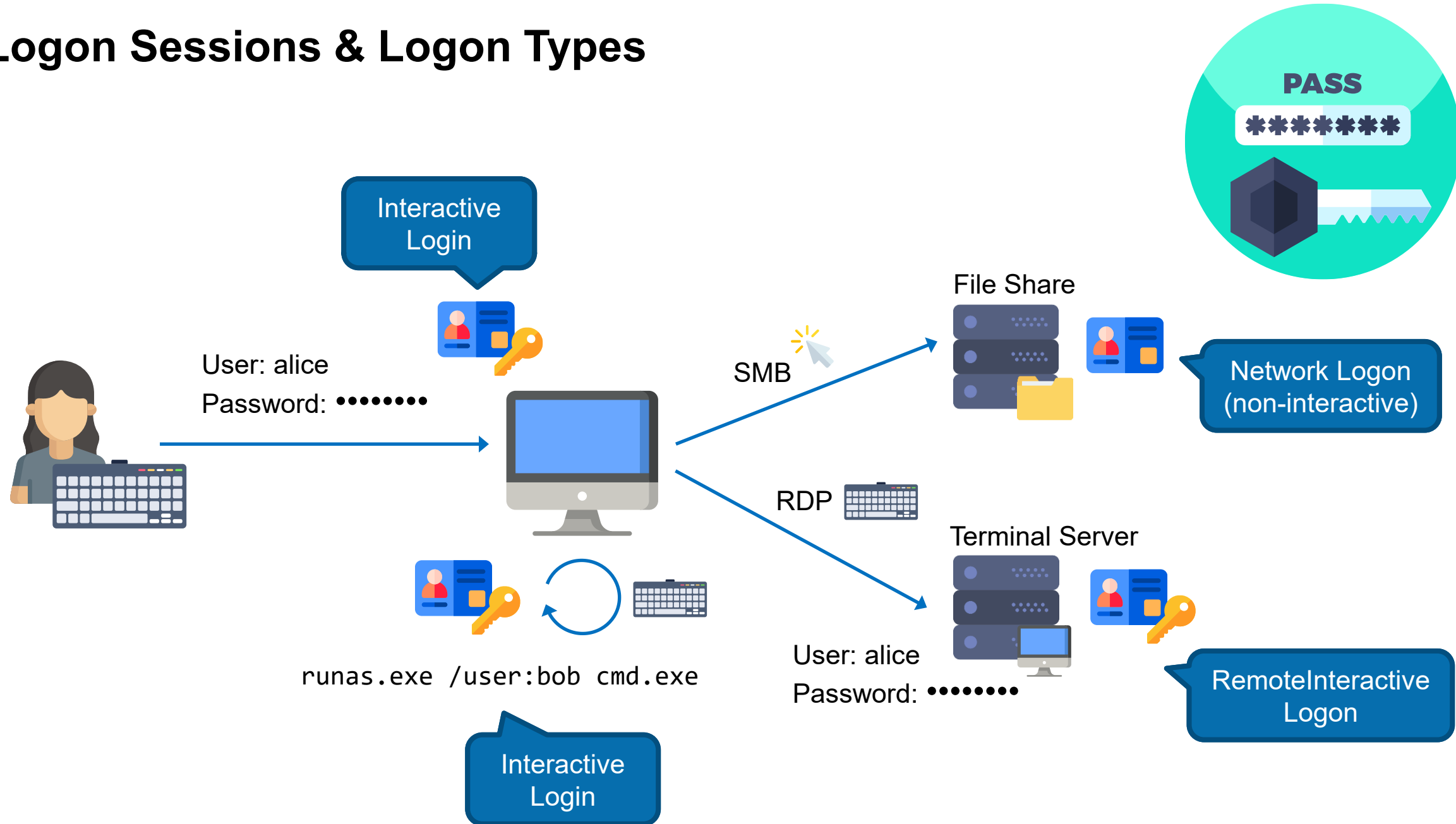


Logon Sessions & Login Types

- A new logon session is created after a successful login
- A user can have more than one logon session
- **Interactive Login**
 - User does specify credentials
 - Credentials are cached within LSA process
 - Used for single sign-on (SSO)
- **Non-Interactive Login / Network Logon**
 - User does not specify credentials
 - Cached credentials from LSA are used (SSO experience e.g. via NTLM/Kerberos)
 - Therefore, only works after interactive login
 - Credentials are not cached on the target because they are not sent to remote system



Logon Sessions & Logon Types



Cached Credentials



- Credentials are cached on the system (they **are** on the system)
 - Can be used for future logins (SSO experience)
- Attacker with local admin privileges might get these credentials (dump LSASS process)
- There are protection mechanisms in place
 - Virtualization Based Security / Credential Guard: LSASS process memory can't be dumped
 - Protected Processes Light (PPL): Handle to process have limited access
 - ASR Rule "Block credential stealing from the Windows LSASS"
 - EDR's detect/block access to the LSASS process
- Lateral movement technique: Instead of dumping the credentials, just use them
 - → Be stealthier in your next pentest / red teaming engagement
 - → Bypass LSASS dumping restrictions

Login Types

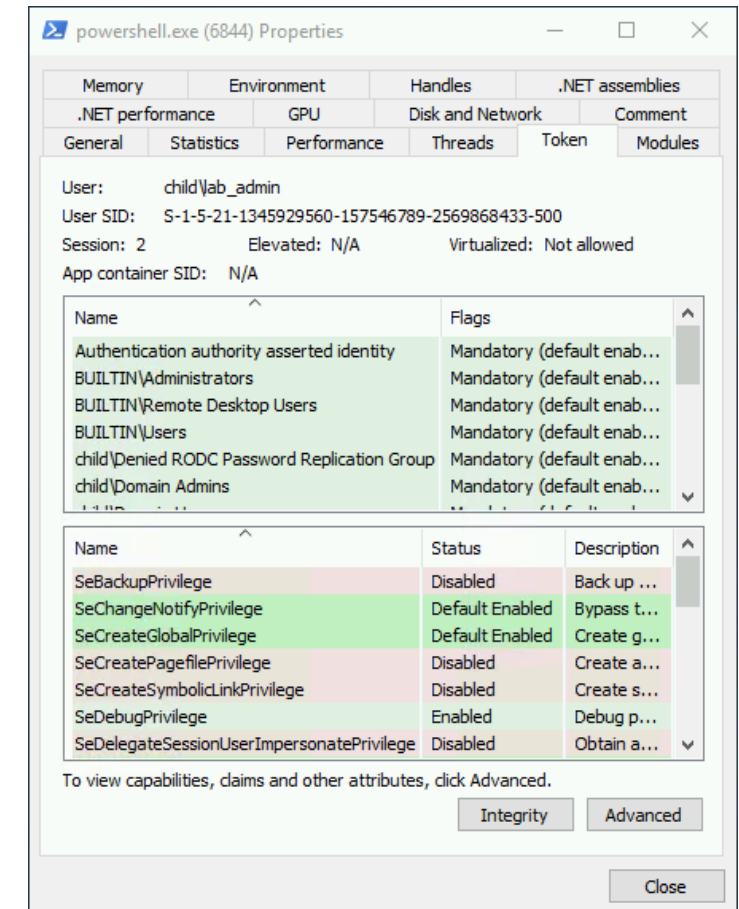
Name	No.	Method	Cached	Example	Comment
Interactive	2	Password SmartCard	Yes	Console / Physical Access runas.exe Psexec with password	
Network (non-interactive)	3	Password NTLM Kerberos	No	SMB & RPC Remote MMC / Registry PowerShell WinRM PsExec without password Some vulnerability scanners	Exception for Kerberos delegation
Batch	4	Password	Yes	Scheduled Tasks	Stored as LSA secret
Service	5	Password	Yes	Windows Services	Stored as LSA secret
NetworkCleartext	8	Password	Yes	IIS Basic Auth PowerShell WinRM with CredSSP	
NewCredentials	9	Password	Yes	runas.exe /netonly rubeus.exe createnetonly CobaltStrike make_token	Current LSA session for local access, new credentials for network access
RemoteInteractive	10	Password SmartCard	Yes	RDP	

<https://learn.microsoft.com/en-us/windows-server/identity/securing-privileged-access/reference-tools-logon-types>

Access Tokens



- Protected object that contains local security context of a user
- Linked to logon session (may or may not have cached credentials)
- Associated to a process or thread
- A user can have more than one access token
- Contains information like
 - User SID
 - Primary Group SID
 - Groups SIDs
 - Integrity Level (medium/high)
 - Logon Session
 - Privileges
 - Token Type (primary or impersonation)
 - Impersonation Level (only for impersonation tokens)
- Used for access control via DACL

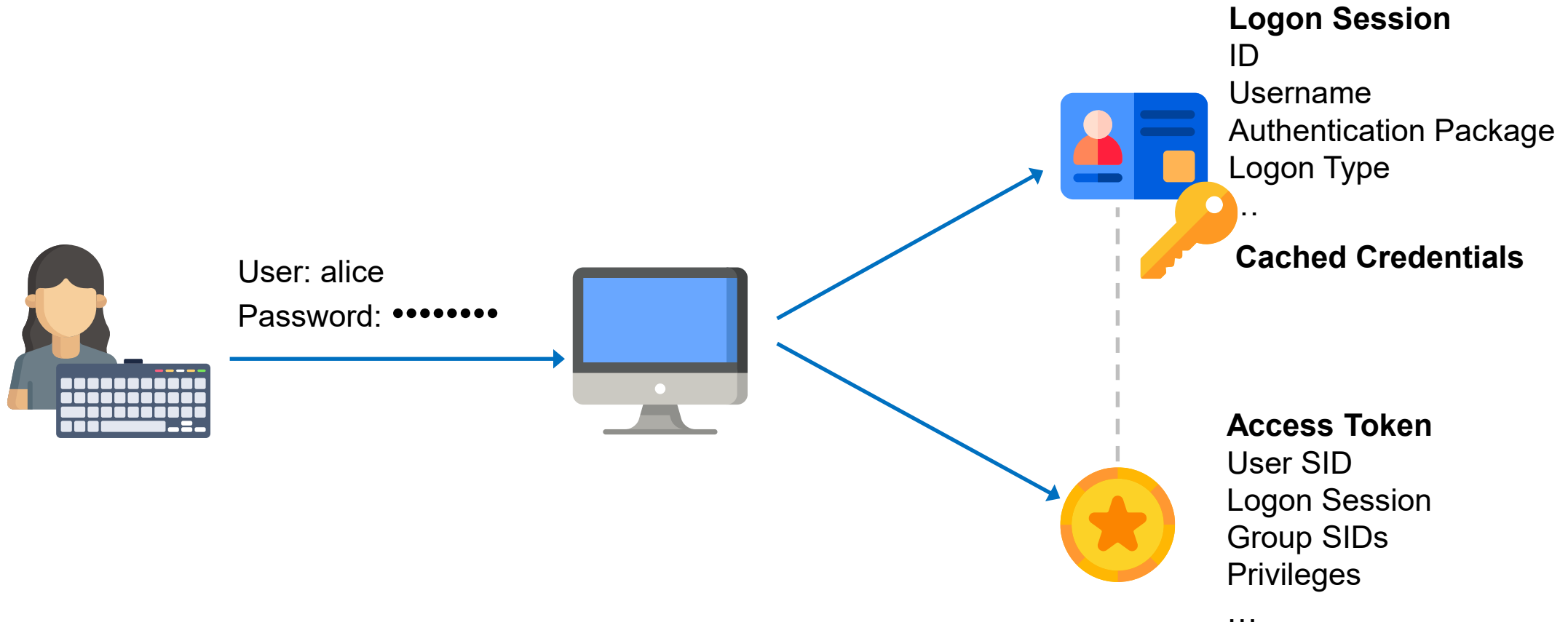


Access Token Types

- **Primary Tokens** (Process Tokens)
 - Created after interactive login
 - Associated to a process, inherited from parent process
- **Impersonation Tokens** (Thread Tokens)
 - Created after non-interactive login / network login
 - Associated to a thread
 - Used for threads to run with different security context process (another token)
 - SMB server runs as SYSTEM but can impersonate authenticated user in new thread
 - Thread is restricted to the permissions of impersonated user (ACLs)
 - Used to enforce access control

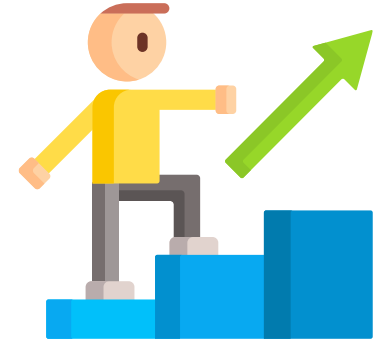


Authentication

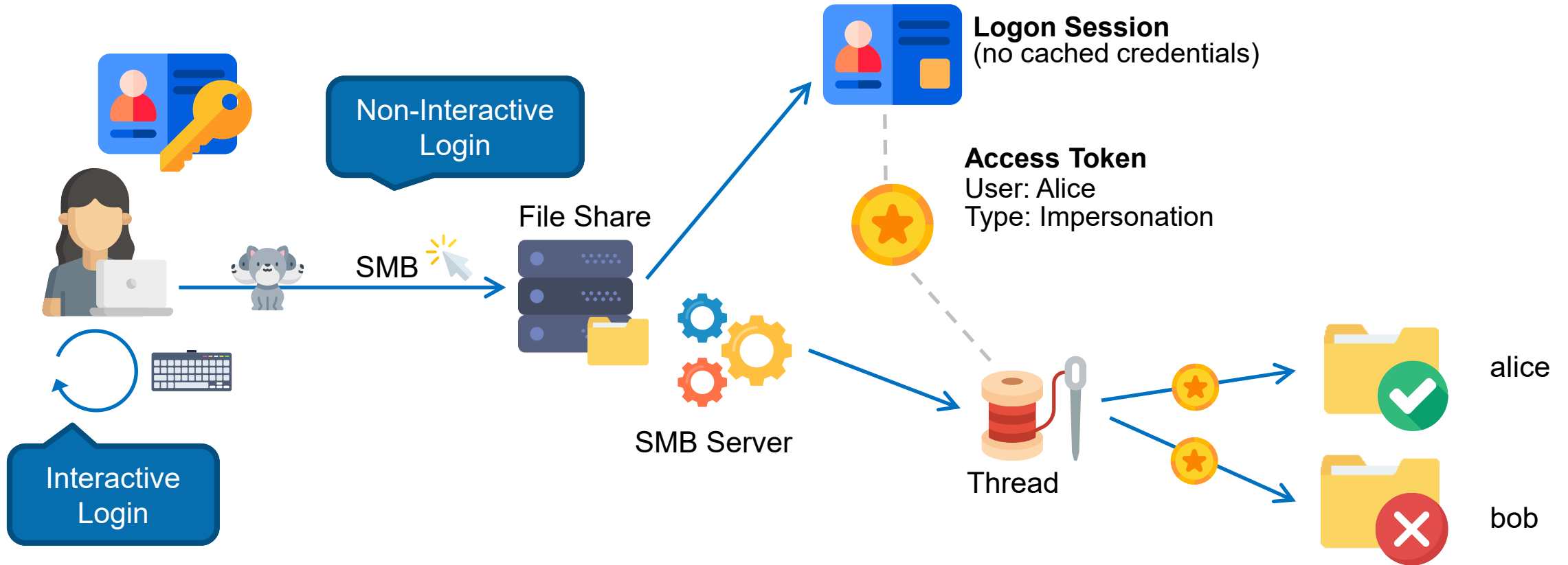


Impersonation Levels

- **Impersonation tokens** have an impersonation level
- **Anonymous**: Threat cannot impersonate user
- **Identification**: Threat can identify user but not impersonate
- **Impersonation**: Threat can impersonate user's security context on local system
- **Delegation**: Threat can impersonate user's security context on local and remote systems
 - Threat can also use cached credentials to authenticate on remote systems

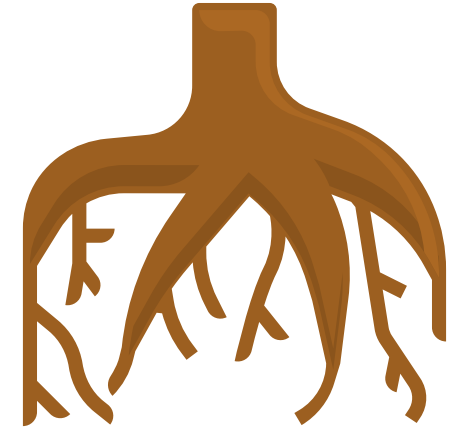


Impersonation Token



User Impersonation

- Such an attack requires privileges
- Normal users
 - Can only see their own access tokens
 - Can use obtained access token in the current process
- High privileged users
 - Can steal any token
 - Can inject into any process
 - Can use obtained access token in a new process
 - Some functionalities even require SYSTEM privileges



User Impersonation

Process Injection

Process Injection



- Inject payload / code into process of other user
- Requires admin permissions
- Common technique used by C2 frameworks

Simplified code:

```
p = Process.GetProcessById(2305); // Open process handle
byte[] code = "...";           // Our shellcode to execute
```

```
var m = VirtualAllocEx(p.Handle, ..., Mode.ReadWrite, ...); // Allocate memory
WriteProcessMemory(p.Handle, m, code, ...);                // Copy code to execute
VirtualProtectEx(p.Handle, m, ..., Mode.ExecuteRead, ...); // Make executable
```

```
CreateRemoteThread(p.Handle, IntPtr.Zero, 0, m, ...); // Execute injected code
```

Metasploit Process Injection

```
meterpreter > getuid
```

```
Server username: FS1\backdoor
```

```
meterpreter > ps -U ffast
```

PID	PPID	Name	Arch	Session	User
---	----	----	----	-----	----
[...]					
7484	4600	powershell.exe	x64	3	child\ffast

```
meterpreter > migrate 7484
```

```
[*] Migrating from 5176 to 7484...
```

```
[*] Migration completed successfully.
```

```
meterpreter > getuid
```

```
Server username: child\ffast
```

User Impersonation

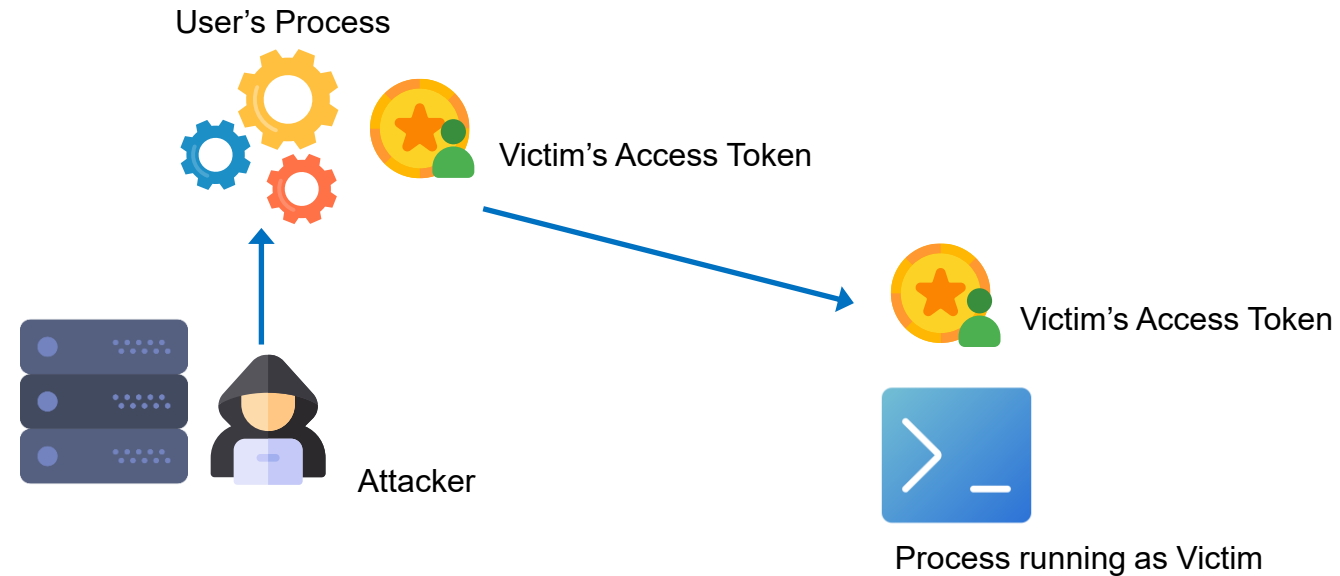
Token Impersonation

Token Impersonation

- Idea: Steal and use an access token of another user
- Requires admin privileges

- Idea / Process

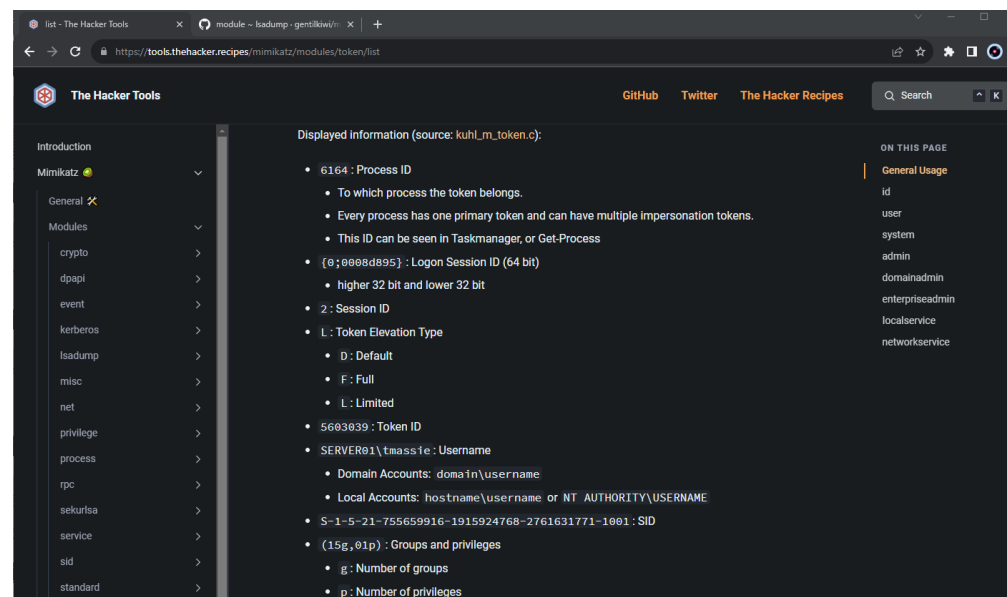
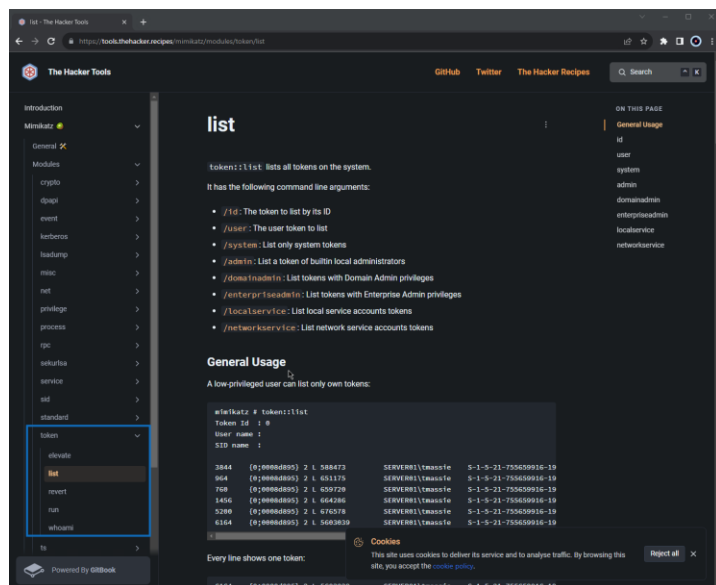
- Open process handle
- Read token information
- Duplicate token
- Create new process using this token



- If logon session has cached credentials, it's possible to authenticate to remote resources
- Otherwise, it's not possible (double hop problem)

Mimikatz Documentation

- The official Mimikatz documentation is not that extensive and detailed
- Only some modules are partially documented
- Sometimes the documentation is a screenshot on Twitter X
- The real documentation is the source 😊
- I added the token module documentation to the “The Hacker Tools” documentation project
- <https://tools.thehacker.recipes/mimikatz/modules/token>



Mimikatz: List Tokens

- Example token

```
6164 {0;0008d895} 2 L 5603039 SERVER01\tmassie  
S-1-5-21-75[...]1771-1001 (15g,01p) Impersonation (Impersonation)
```

- **6164**: Process ID
 - To which process the token belongs
 - Every process has one primary token and can have multiple impersonation tokens
 - This ID can be seen in Taskmanager, tasklist.exe, or Get-Process
- **{0;0008d895}**: Logon Session ID (64 bit)
 - higher 32 bit and lower 32 bit
- **2**: Session ID
 - Terminal session, one per interactive login

Mimikatz: List Tokens

- Example token

```
6164 {0;0008d895} 2 L 5603039 SERVER01\tmassie  
S-1-5-21-75[...]1771-1001 (15g,01p) Impersonation (Impersonation)
```

- L: Token Elevation Type
 - D: Default (low-privileged process)
 - F: Full (high-privileged process)
 - L: Limited (process started as admin but admin privileges removed)
- 5603039: Token ID
- SERVER01\tmassie: Username
 - Domain Accounts: domain\username
 - Local Accounts: hostname\username or NT AUTHORITY\USERNAME
- S-1-5-21-75[...]1771-1001: SID

Mimikatz: List Tokens

- Example token

```
6164 {0;0008d895} 2 L 5603039 SERVER01\tmassie  
S-1-5-21-75[...]1771-1001 (15g,01p) Impersonation (Impersonation)
```

- (15g,01p): Groups and privileges
 - g: Number of groups
 - p: Number of privileges
- Impersonation: Token Type
 - Unknown
 - Primary
 - Impersonation

Mimikatz: List Tokens

- Example token

```
6164 {0;0008d895} 2 L 5603039 SERVER01\tmassie  
S-1-5-21-75[...]1771-1001 (15g,01p) Impersonation (Impersonation)
```

- (Impersonation): Impersonation Level (Only for impersonation tokens)
 - Anonymous: Server cannot impersonate the client
 - Identification: Server can identify client but not impersonate
 - Impersonation: Server can impersonate client's security context on local system
 - Delegation: Server can impersonate client's security context on remote systems using cached credentials

Mimikatz: List Current Token

- List current token:

```
mimikatz # token::whoami
```

```
* Process Token : {0;0030f129} 4 F 38912331      FS1\backdoor      S-1-5-[...]01  
(15g,24p)      Primary
```

```
* Thread Token  : no token
```

- Visible information:
 - Process token which will be used for newly created processes
 - Impersonation token which will be used for newly created threads
(Process token will be used if no thread token is listed. By default, there is no thread token.)

Mimikatz: List Current Token Details

```
mimikatz # token::whoami /full
* Process Token : {0;0030f129} 4 F 41407048
FS1\backdoor S-1-5-[...]01 (15g,24p)
Primary
  G:[MDE ] FS1\None
  G:[MDE ] Everyone
  G:[MDE ] NT AUTHORITY\Local account and
             member of Administrators group
  G:[MDE ] BUILTIN\Users
  G:[MDEO ] BUILTIN\Administrators
[...]
  G:[MDE ] LOCAL
  G:[MDE ] NT AUTHORITY\NTLM Authentication
  G:[ ] Mandatory Label\High Mandatory Level
[...]
  P:[ ] SeIncreaseQuotaPrivilege
  P:[ ] SeShutdownPrivilege
  P:[ E ] SeDebugPrivilege
  P:[ ] SeSystemEnvironmentPrivilege
  P:[DE ] SeChangeNotifyPrivilege
  P:[ ] SeRemoteShutdownPrivilege
[...]
* Thread Token : no token
```

- *: Token Type
 - Process Token: Primary Token
 - Thread Token: Impersonation Token
- G: Assigned groups
 - M: Mandatory
 - D: Enabled by Default
 - E: Group Enabled
 - O: Group Owner
 - U: Group Use for Deny Only
 - L: Group Logon ID
 - R: Group Resource
- P: Privilege Information
 - D: Privilege Enabled by Default
 - E: Privilege Enabled
 - R: Privilege Removed
 - A: Privilege used for Access

Demo

PS C:\>



Type here to search



DEU

8:04 AM
8/8/2025



PS C:\> █



🔍 Type here to search



DEU

8:12 AM
8/8/2025



No new notifications

mimikatz #



Type here to search



DEU

8:31 AM
8/8/2025



References

References

- Security Implications of Windows Access Tokens – A Penetration Tester's Guide. Luke Jennings. MWR InfoSecurity. April 2008.
 - Paper: <https://www.exploit-db.com/docs/english/13054-security-implications-of-windows-access-tokens.pdf>
 - Talk "One Token to Rule Them All" @ 24c3: https://media.ccc.de/v/24c3-2235-en-one_token_to_rule_them_all
- Detecting Access Token Manipulation. William Burgess. BlackHat USA 2020.
 - Description: <https://www.blackhat.com/us-20/briefings/schedule/#detecting-access-token-manipulation-20524>
 - Talk: <https://www.youtube.com/watch?v=RMVyYvt0bLY>
 - Slides: <http://i.blackhat.com/USA-20/Thursday/us-20-Burgess-Detecting-Access-Token-Manipulation.pdf>

References

- Abusing Windows' tokens to compromise Active Directory without touching LSASS. Aurelien Chalot. Orange. October 2022.
 - Blogpost: <https://sensepost.com/blog/2022/abusing-windows-tokens-to-compromise-active-directory-without-touching-lsass/>
 - Tool: Impersonate: <https://github.com/sensepost/impersonate>
 - Impersonate code: <https://github.com/sensepost/impersonate/blob/main/Impersonate/Impersonate/Impersonate.cpp>
- Token Impersonation in C#. Rasta Mouse. December 2022.
 - Blog: <https://rastamouse.me/token-impersonation-in-csharp/>

References

- Understanding Windows Lateral Movements (Remake!). Daniel López Jiménez. April 2023.
 - Slides: https://attl4s.github.io/assets/pdf/Understanding_Windows_Lateral_Movements_2023.pdf
 - Video 1: Understanding Windows Lateral Movements - Windows Authentication: https://www.youtube.com/watch?v=OjWn4VUeT8A&list=PLwb6et4T42ww94O3z5QDNQsO1f_BwhX-L
 - Video 2: Understanding Windows Lateral Movements - Access Token Manipulation & Passwords: https://www.youtube.com/watch?v=J0c9HNEeHbE&list=PLwb6et4T42ww94O3z5QDNQsO1f_BwhX-L
 - Video 3: Understanding Windows Lateral Movements - Hashes, Tickets and the SSPI: https://www.youtube.com/watch?v=plNq51wQB-w&list=PLwb6et4T42ww94O3z5QDNQsO1f_BwhX-L

References

- Fortra Cobalt Strike Blog: Windows Access Tokens and Alternate Credentials: <https://www.cobaltstrike.com/blog/windows-access-tokens-and-alternate-credentials/>
- Potatoes - Windows Privilege Escalation. Jorge Lajara. November 2022. https://jlajara.gitlab.io/Potatoes_Windows_Privesc
- MITRE ATT&CK: Access Token Manipulation (T1134) <https://attack.mitre.org/techniques/T1134/>
 - Access Token Manipulation: Token Impersonation/Theft (T1134.001): <https://attack.mitre.org/techniques/T1134/001/>
 - Access Token Manipulation: Create Process with Token (T1134.002): <https://attack.mitre.org/techniques/T1134/002/>
 - Access Token Manipulation: Make and Impersonate Token (T1134.003): <https://attack.mitre.org/techniques/T1134/003/>
 - Access Token Manipulation: Parent PID Spoofing (T1134.004): <https://attack.mitre.org/techniques/T1134/004/>
 - Access Token Manipulation: SID-History Injection (T1134.005): <https://attack.mitre.org/techniques/T1134/005/>

References

- Microsoft Documentation
 - Access Tokens: <https://learn.microsoft.com/en-us/windows/win32/secauthz/access-tokens>
- Windows API
 - NtQuerySystemInformation: <https://learn.microsoft.com/en-us/windows/win32/api/winternl/nf-winternl-ntquerysysteminformation>
- Tools
 - <https://hunter2.gitbook.io/darthsidious/privilege-escalation/token-impersonation>
 - Metasploit Incognito Module
 - Fun with Incognito. Offsec. <https://www.offsec.com/metasploit-unleashed/fun-incognito/>

